

Name: Riwaz Shrestha

UID: U23420865

Collaborators: Denali Schlesinger

Problem 1:

Code:

```
4  fn main() {
5      let mut pyramid1: Shape = Shape::create(shape: "Pyramid", l: 2.0, w: Some(2.0), h: Some(3.0));
6      println!("pyramid1: {:?}", pyramid1);
7      println!("pyramid1 volume: {}", pyramid1.volume());
8      println!("pyramid1 surface area: {}", pyramid1.surface_area());
9      pyramid1.double();
10     println!("pyramid1: {:?}", pyramid1);
11     println!("pyramid1 has the correct parameters: {}", pyramid1.verify());
12
13     println!();
14
15     let mut cuboid1: Shape = Shape::create(shape: "Cuboid", l: 2.0, w: Some(2.0), h: Some(3.0));
16     println!("cuboid1: {:?}", cuboid1);
17     println!("cuboid1 volume: {}", cuboid1.volume());
18     println!("cuboid1 surface area: {}", cuboid1.surface_area());
19     cuboid1.double();
20     println!("cuboid1: {:?}", cuboid1);
21     println!("cuboid1 has the correct parameters: {}", cuboid1.verify());
22
23     println!();
24
25     let mut sphere1: Shape = Shape::create(shape: "Sphere", l: 2.0, w: None, h: None);
26     println!("sphere1: {:?}", sphere1);
27     println!("sphere1 volume: {}", sphere1.volume());
28     println!("sphere1 surface area: {}", sphere1.surface_area());
29     sphere1.double();
30     println!("sphere1: {:?}", sphere1);
31     println!("sphere1 has the correct parameters: {}", sphere1.verify());
32
33     println!();
34     println!("Tests: \n");
35
36     let pyramid2: Shape = Shape::create(shape: "Pyramid", l: -2.0, w: Some(-2.0), h: Some(4.6));
37     println!("pyramid2: {:?}", pyramid2);
38     println!("pyramid2 has the correct parameters: {}", pyramid2.verify());
39
40     println!();
41
42     let cuboid2: Shape = Shape::create(shape: "Cuboid", l: -1.0, w: Some(2.0), h: Some(4.7));
43     println!("cuboid2: {:?}", cuboid2);
44     println!("cuboid2 has the correct parameters: {}", cuboid2.verify());
45
46     println!();
47
48     let sphere2: Shape = Shape::create(shape: "Sphere", l: -2.0, w: None, h: None);
49     println!("sphere2: {:?}", sphere2);
50     println!("sphere2 has the correct parameters: {}", sphere2.verify());
51
52
53 }
```

This is the main function where I create the different shapes and print out the shape along with the volume, surface_area, double the shape size and a verification function. The most important part of this code is that I made the instances where I used the double function mutable. I used Some(x) for the last two variables of the pyramids and cuboids since I had them as option f64 for the create function. I used None for the last two variables of the spheres so that the option enum returned None and did not fail due to not having the right number of inputs.

```

55 #[derive(Debug)]
56 2 implementations
57 enum Shape {
58     Pyramid(f64, f64, f64),
59     Cuboid(f64, f64, f64),
60     Sphere(f64),
61 }
62 impl Shape {
63     fn create(shape: &str, l: f64, w: Option<f64>, h: Option<f64>) -> Self {
64         match shape {
65             "Pyramid" => Self::Pyramid(l, w.unwrap(), h.unwrap()),
66             "Cuboid" => Self::Cuboid(l, w.unwrap(), h.unwrap()),
67             "Sphere" => Self::Sphere(l),
68             _ => todo!("Invalid shape type: {}", shape),
69         }
70     }
71     fn volume(&self) -> f64 {
72         match self {
73             Shape::Pyramid(l: &f64, w: &f64, h: &f64) => (*l * *w * *h) / 3.0,
74             Shape::Cuboid(l: &f64, w: &f64, h: &f64) => *l * *w * *h,
75             Shape::Sphere(r: &f64) => (4.0 / 3.0) * PI * r.powi(2),
76         }
77     }
78     fn surface_area(&self) -> f64 {
79         match self {
80             Shape::Pyramid(l: &f64, w: &f64, h: &f64) => (*l * *w) + l * ((*w / 2.0).powi(2) + h.powf(2.0)).sqrt() + *w * ((*l / 2.0).powi(2) + h.powf(2.0)).sqrt(),
81             Shape::Cuboid(l: &f64, w: &f64, h: &f64) => (2.0 * *l * *w) + (2.0 * *w * *h) + (2.0 * *l * *h),
82             Shape::Sphere(r: &f64) => 4.0 * PI * r.powi(2),
83         }
84     }
85     fn double(&mut self) {
86         match self {
87             Shape::Pyramid(l: &mut f64, w: &mut f64, h: &mut f64) => {
88                 *l *= 2.0;
89                 *w *= 2.0;
90                 *h *= 2.0;
91             },
92             Shape::Cuboid(l: &mut f64, w: &mut f64, h: &mut f64) => {
93                 *l *= 2.0;
94                 *w *= 2.0;
95                 *h *= 2.0;
96             },
97             Shape::Sphere(r: &mut f64) => {
98                 *r *= 2.0;
99             },
100         }
101     }
102     fn verify(&self) -> bool {
103         match self {
104             Shape::Pyramid(l: &f64, w: &f64, h: &f64) => *l > 0.0 && *w > 0.00 && *h > 0.00,
105             Shape::Cuboid(l: &f64, w: &f64, h: &f64) => *l > 0.0 && *w > 0.00 && *h > 0.00,
106             Shape::Sphere(r: &f64) => *r > 0.0,
107         }
108     }
109 } impl Shape

```

This is the part of the code where I created the enum “Shape” with tuples. I used chatGPT in this section to figure out the difference between tuple-type enums and struct-type enums. The main idea I got that applied to my circumstance was that the tuple type only stored values with no assigned variable while the struct type stored them in labeled variables.

In the impl “Shape” I wrote the “create” function with 4 inputs. A string to identify which enum to use, a f64, and two option f64 to account for the difference in input variables for the different enums. The option f64 returns either some() or none(). I had this function return Self, which created an instance of the enum. I had to use the unwrap function to get the values inside of the variables when they were created and I had to use some(x) when creating the shapes since I used the option f64. I used a match statement that matched the string variable and input the rest of the variables to create an instance of the shape. I added an error handler so that the code would not fail if an unacceptable string was input.

For the volume, surface area, and verify functions I used a match statement to match to &self, a copy of the instance, and returned the corresponding volume, surface area, and verify results. Volume and surface area returned f64 while verify returned a boolean corresponding to whether the shape had acceptable dimensions.

The function “double” took in a mutable version of the instance and used `*` to dereference and scale the values by a factor of 2. The previous functions also used `&self` to reference the current implementation and `*` to dereference the variables and use the values. This function returns nothing as it is meant to modify the current instance of the shape.

Output:

```

  TERMINAL  PROBLEMS  DEBUG CONSOLE  OUTPUT  PORTS
● (base) riwazshrestha@crc-dot1x-nat-10-239-2-183 HW3 % cd q1
● (base) riwazshrestha@crc-dot1x-nat-10-239-2-183 q1 % cargo run
  Compiling q1 v0.1.0 (/Users/riwazshrestha/Desktop/DS210/Homework/HW3/q1)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.31s
  Running `target/debug/q1`
pyramid1: Pyramid(2.0, 2.0, 3.0)
pyramid1 volume: 4
pyramid1 surface area: 16.64911064067352
pyramid1: Pyramid(4.0, 4.0, 6.0)
pyramid1 has the correct parameters: true

cuboid1: Cuboid(2.0, 2.0, 3.0)
cuboid1 volume: 12
cuboid1 surface area: 32
cuboid1: Cuboid(4.0, 4.0, 6.0)
cuboid1 has the correct parameters: true

sphere1: Sphere(2.0)
sphere1 volume: 16.755160819145562
sphere1 surface area: 50.26548245743669
sphere1: Sphere(4.0)
sphere1 has the correct parameters: true

Tests:

pyramid2: Pyramid(-2.0, -2.0, 4.6)
pyramid2 has the correct parameters: false

cuboid2: Cuboid(-1.0, 2.0, 4.7)
cuboid2 has the correct parameters: false

sphere2: Sphere(-2.0)
sphere2 has the correct parameters: false
○ (base) riwazshrestha@crc-dot1x-nat-10-239-2-183 q1 %
```

For each Shape, I printed the shape information, volume, surface area, the shape doubled, and if the shape was valid or not. Valid referring to if all the side lengths of the shape were greater than 0. I tested that the verify function worked by creating one of each shape with a negative value.

Problem 2:

Code:

```
1 use std::f64::consts::PI;
2
3
4 ▶ Run | e Debug
5 fn main() {
6     let sides_list: [u32;10] = [4, 8, 16, 32, 64, 128, 256, 512, 2048, 65536];
7     let length_list: [f64;3] = [2.0,5.0,10.0];
8     for side: &u32 in &sides_list {
9         for length: &f64 in &length_list {
10             let polygon: Polygon = Polygon{
11                 sides: *side, lengths: *length;
12             };
13             let poly_per: f64 = polygon.perimeter();
14             let poly_area: f64 = polygon.area();
15             let poly_rad: f64 = polygon.radius();
16             let poly_apo: f64 = polygon.apothem();
17             let cir_area: f64 = PI * poly_rad.powi(2);
18
19             println!("{:?} \n perimeter: {} \n area: {} \n radius: {} \n apothem: {} \n circle area: {} ",
20                 polygon, poly_per, poly_area, poly_rad, poly_apo, cir_area);
21             println!("{}", "");
22         }
23     }
24 }
25
26 1 implementation
27 trait Functions {
28     fn area(&self) -> f64;
29     fn perimeter(&self) -> f64;
30     fn radius(&self) -> f64;
31     fn apothem(&self) -> f64;
32 }
33
34 #[derive(Debug)]
35 2 implementations
36 struct Polygon {
37     sides: u32,
38     lengths: f64,
39 }
40
41 //run an implementation
42
43 impl Functions for Polygon {
44     fn area(&self) -> f64 {
45         (self.sides as f64 * self.lengths.powi(2)) / (4.0 * (PI / self.sides as f64).tan())
46     }
47     fn perimeter(&self) -> f64 {
48         self.sides as f64 * self.lengths
49     }
50     fn radius(&self) -> f64 {
51         self.lengths / (2.0 * (PI / self.sides as f64).sin())
52     }
53     fn apothem(&self) -> f64 {
54         self.lengths / (2.0 * (PI / self.sides as f64).tan())
55     }
56 }
```

I used PI from the standard library.

In the main function, I created two arrays, one with the given values, and another with a list of lengths for the sides of the polygons. The sides list was u32 and the lengths list was f64. Since I am using two differing types I made sure to change the u32 to f64 whenever it was required. I then ran a for loop, iterating the values in the length list. I ran another for loop, iterating the values of the sides list. I used a nested loop to get every possible combination between the two lists. I then created a polygon using the sides and side lengths and ran each function for the instance of the polygon. I then calculated the area of a circle with the radius corresponding to the radius of the polygon. Lastly, I printed all of the information calculated.

In the trait named Functions, I initialized 4 functions that all returned f64.

I used derive debug so that I could print the polygons I made without running into any errors.

I created a struct Polygon, with the variables “sides” and “lengths”.

I implemented the trait Functions for the struct Polygon. I needed to cover all the functions inside the trait so I created 4 corresponding functions returning the area, perimeter, radius, and apothem of the instance of the polygon. They all took in &self to refer to the current instance.

Output:

```
TERMINAL  PROBLEMS  DEBUG CONSOLE  OUTPUT  PORTS

, radius: 407.446881033519
, apothem: 407.41620103273084
, circle area: 521545.0982508518

Polygon { sides: 512, lengths: 2.0 }
  perimeter: 1024
  area:83441.97960358398
, radius: 162.9756843844514
, apothem: 162.97261641324997
, circle area: 83444.07400920018

Polygon { sides: 512, lengths: 5.0 }
  perimeter: 2560
  area:521512.3725223999
, radius: 407.4392109611285
, apothem: 407.4315410331249
, circle area: 521525.46255750104

Polygon { sides: 512, lengths: 10.0 }
  perimeter: 5120
  area:2086049.4900895995
, radius: 814.878421922257
, apothem: 814.8630820662498
, circle area: 2086101.8502300042

Polygon { sides: 2048, lengths: 2.0 }
  perimeter: 4096
  area:1335087.3816625027
, radius: 651.8989025679382
, apothem: 651.8981355773939
, circle area: 1335089.4760582622

Polygon { sides: 2048, lengths: 5.0 }
  perimeter: 10240
  area:8344296.135390641
, radius: 1629.7472564198454
, apothem: 1629.7453389434845
, circle area: 8344309.225364137

Polygon { sides: 2048, lengths: 10.0 }
  perimeter: 20480
  area:33377184.541562565
, radius: 3259.4945128396907
, apothem: 3259.490677886969
, circle area: 33377236.90145655

Polygon { sides: 65536, lengths: 2.0 }
  perimeter: 131072
  area:1367130550.1056657
, radius: 20860.75670893039
, apothem: 20860.75668496194
, circle area: 1367130552.2000606

Polygon { sides: 65536, lengths: 5.0 }
  perimeter: 327680
  area:8544565938.160411
, radius: 52151.89177232597
, apothem: 52151.89171240485
, circle area: 8544565951.2503805

Polygon { sides: 65536, lengths: 10.0 }
  perimeter: 655360
  area:34178263752.641644
, radius: 104303.78354465195
, apothem: 104303.7834248097
, circle area: 34178263805.001522

(base) riwazshrestha@crc-dot1x-nat-10-239-2-183 q2 %
```

I output the instance of the polygon, perimeter, area, radius, apothem, and the area of a circle with the same radius for each combination of values in the two lists using a nested loop.

The relationship between the area of the polygon and the circle as the number of sides increases is that the area of the polygon gets closer and closer to the area of the circle but stays less than the area of the circle as the circle uses the radius which leads to a circumscribed circle. The circle could surround the outside of the polygon.

As the number of sides increases, the two areas get more similar. Since the circle uses the radius, it is a circumscribed circle meaning that it will always be larger than the polygon.